T4 Exercises – Fateme Firouznia

1. **The command netstat -nat lists all current TCP connections. How can I count the number of connections in each state?**

   The netstat -nat command displays active TCP sockets along with their connection states. Since the command itself only reports raw data, the number of connections in each TCP state must be calculated by processing its output.

   The state information is taken from the State column, after removing the header lines. The extracted values are then sorted so identical states are grouped together, and finally the occurrences of each state are counted using standard command-line tools.

   > For example, this can be done by combining awk, sort, and uniq -c, which allows counting how many connections are in states such as ESTABLISHED, LISTEN, or TIME_WAIT.

2. **How can we read the stdin, stdout, and stderr streams of a running process using the cat command?**

   In Linux, the standard input, output, and error streams of a running process are exposed as file descriptors. Each process has a directory under /proc identified by its PID, and the file descriptors are available in /proc/<PID>/fd/.

   File descriptor 0 corresponds to stdin, 1 to stdout, and 2 to stderr. Since these descriptors are represented as files, they can be read using the cat command, provided sufficient permissions are available. By accessing the appropriate descriptor file, it is possible to inspect the input or output streams of a running process.

3. **What is an inode and what is its purpose?**

   An inode is a filesystem data structure that stores metadata about a file, including permissions, ownership, size, timestamps, and the locations of the file's data blocks on disk. File names are not stored in the inode; instead, directories map file names to inode numbers. This allows the filesystem to manage and reference files efficiently.

4. **When a new file is created using the touch command, what happens step-by-step in the filesystem, especially in terms of the inode?**

   When a new file is created using the touch command, the filesystem allocates a new inode for the file. The inode is initialized with metadata such as timestamps and

permissions. Then, a new directory entry is created that links the file name to the allocated inode. At this point, the file exists without data content, but its inode and directory reference are in place.

5. **What does the stat command show about a file's inode, and how should we interpret that information?**

   The stat command displays inode-related information about a file, such as the inode number, permissions, ownership, file size, link count, and access, modification, and change timestamps. This information helps interpret how the file is managed within the filesystem. For example, the link count indicates how many directory entries reference the same inode, and the timestamps show different types of file activity.

6. **When using the mv command, what changes occur at the inode level? Under what conditions does the inode stay the same or change?**

   When the mv command is used within the same filesystem, the file’s inode remains unchanged and only the directory entry is updated to reflect the new name or location. However, when a file is moved across different filesystems, a new inode is created on the destination filesystem, and the file data is copied, resulting in a different inode.

7. **When you run ls, how does it retrieve the names of all files and directories in the current directory?**

   When the ls command is executed, it reads the contents of the current directory from the filesystem. A directory contains entries that map file and directory names to their corresponding inode numbers. The ls command retrieves these directory entries and displays the names of the files and directories.

8. **When you run the top command, it shows process states such as running, sleeping, zombie, and stopped. Can you explain what each state means and when it occurs?**

   In the top command, each process is shown in a specific state.
   A process in the **Running** state is currently executing on the CPU or ready to run.
   A **Sleeping** process is waiting for an event or resource, such as I/O.
   A **Zombie** process has finished execution but its parent has not yet collected its exit status.
   A **Stopped** process has been halted temporarily, usually by a signal or user action.

9. **If your server shows a high load average and a high number of context switches, what could this indicate?**

   A high load average combined with a high number of context switches usually indicates heavy contention for system resources. This can be caused by many active processes competing for CPU time or by processes frequently blocking on I/O. As a result, the system spends more time switching between processes rather than executing useful work.

10. **Try running atop, htop, and nmon. Where do these tools retrieve their data from?**

    Tools such as atop, htop, and nmon retrieve their data directly from the Linux kernel. They read system and process information from kernel interfaces like /proc and /sys, which expose real-time data about CPU usage, memory, processes, disk activity, and network statistics.